

## CSCE 735 PARALLEL COMPUTING · MODULE 05 · LECTURE NOTES

# Lock-Free Programming

*Progress guarantees, and whether you need them.*

## MODULE OUTCOMES

- Define the progress guarantees precisely.
- Implement a compare-and-swap loop.
- Explain the ABA problem and its solutions.
- Explain safe memory reclamation.
- Judge honestly when lock-free is warranted.

## 1 Progress guarantees

| Guarantee               | Definition                                                                                                      | Assessment                                                                                                                                                            |
|-------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Wait-free</b>        | Every thread completes its operation in a bounded number of its own steps, regardless of what other threads do. | <b>The strongest guarantee, and rare in practice.</b> Required in hard real-time systems. Wait-free versions of common structures exist and are substantially slower. |
| <b>Lock-free</b>        | <b>Some</b> thread makes progress in a bounded number of steps. An individual thread may retry indefinitely.    | System-wide progress is guaranteed; individual starvation is not excluded. <b>This is what most 'lock-free' structures provide.</b>                                   |
| <b>Obstruction-free</b> | A thread running in isolation completes in a bounded number of steps.                                           | The weakest non-blocking guarantee. Permits livelock when threads contend; requires a contention manager in practice.                                                 |
| <b>Blocking</b>         | A thread suspended while holding a lock can prevent all others from progressing.                                | <b>What a mutex provides.</b> Entirely adequate for most purposes.                                                                                                    |

**LOCK-FREE IS A LIVENESS PROPERTY, NOT A SPEED CLAIM**

**The name is the problem.** 'Lock-free' sounds like a performance claim and is a *liveness* claim: it guarantees that some thread makes progress even if others are suspended arbitrarily — descheduled, page-faulted, or terminated.

**It says nothing about throughput.** A lock-free structure under heavy contention can perform considerably worse than a mutex-protected one, because every failed compare-and-swap discards work that was already done, and the retries add load that causes more failures.

**Where the guarantee genuinely matters:** hard real-time systems where a priority-inverted thread holding a lock is unacceptable; signal handlers, which cannot safely take a lock that the interrupted code may hold; and any context where a thread might be stopped and never resumed.

**If what you want is speed, measure.** The answer is frequently that a well-placed mutex wins, and more frequently still that restructuring to avoid the sharing wins by more than either (§4).

## 2 Compare and swap

```
template <class F>
void atomic_update(std::atomic<int>& v, F f) {
    int old = v.load(std::memory_order_relaxed);
    int desired;
    do {
        desired = f(old);
    } while (!v.compare_exchange_weak(old, desired,
                                     std::memory_order_release,    // success
                                     std::memory_order_relaxed));    // failure
    // compare_exchange_weak UPDATES `old` on failure, so the loop
    // automatically retries against the fresh value.
}
```

Use `compare_exchange_weak` inside a loop and `compare_exchange_strong` outside one. The weak form is permitted to fail spuriously — to report a mismatch when the values in fact matched — which allows a cheaper implementation on load-linked/store-conditional architectures such as ARM and POWER, where an intervening cache event can invalidate the reservation. Inside a loop a spurious failure simply causes another iteration, so the cheaper form is free; outside a loop it would be a bug.

### THE ABA PROBLEM

**Compare-and-swap checks that a value is unchanged. It does not check that nothing happened.**

**Thread 1 reads the value A and is descheduled. Thread 2 changes it to B. Thread 3 changes it back to A. Thread 1 resumes and its CAS succeeds,** because the value it compares against does indeed equal A.

**But the world has changed.** In a linked structure, where the value is a pointer, the node at that address may have been popped, freed, and the memory reallocated for something entirely different. Thread 1 proceeds on the assumption that its cached view of the structure is still valid, and it is not.

**Solutions:** a **tagged pointer** that packs a version counter alongside the pointer and increments it on every modification, so the value never genuinely repeats; **double-width CAS**, which swaps pointer and counter atomically; or a **memory reclamation scheme** that prevents the address being reused while anyone might still be looking at it — which is §3, and which is needed anyway.

### 3 Safe memory reclamation

#### YOU CANNOT FREE A NODE SOMEONE MIGHT STILL BE READING

**With a lock, reclamation is trivial.** Holding the lock means no other thread is inside the structure, so a node that has been unlinked can be freed immediately.

**Without one, a thread may be holding a pointer to the node you have just unlinked** — it read the pointer before you removed it, and it has no way of telling you so and no obligation to check.

**Freeing it is a use-after-free**, which is both a correctness catastrophe and a security vulnerability, and among the hardest classes of bug to diagnose because the symptom appears far from the cause.

**This, rather than the algorithm, is what makes lock-free data structures difficult.** Writing a lock-free stack from the published algorithm is an afternoon's work. **Reclaiming its memory correctly is the project**, and it is where the published algorithms were found to have bugs.

| Scheme                        | Mechanism                                                                                                                                                                         | Cost and use                                                                                                                                                                                                                                      |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hazard pointers               | Each thread publishes, in a globally visible slot, the pointers it is currently dereferencing. A thread wishing to free a node first checks that no hazard pointer references it. | <b>A store and a memory fence on every access</b> , which is significant for read-heavy workloads. Bounded memory usage, which is its advantage.                                                                                                  |
| Epoch-based reclamation (RCU) | Threads announce entry to and exit from a read-side critical section. A node may be freed once every thread has passed through a quiescent point since it was unlinked.           | <b>Reads are extremely cheap</b> — essentially free in the kernel variant. Frees are deferred, so memory usage is unbounded if a thread stalls. <b>The standard answer where reads dominate</b> , and it is used pervasively in the Linux kernel. |
| Reference counting            | An atomic counter per node, incremented on acquisition.                                                                                                                           | <b>An atomic operation per access, on shared data</b> — which is exactly the contention the structure was meant to avoid. Simple and frequently the slowest.                                                                                      |
| Never free                    | Leak the nodes, or allocate from a fixed pool that is reused only when the structure is known to be quiescent.                                                                    | <b>More viable than it sounds</b> , and underused. For a bounded structure with a bounded lifetime, or one whose nodes are recycled through a type-stable pool, this removes the entire problem.                                                  |

### 4 Judgement

- 1 Can you avoid sharing altogether?** Per-thread state with a final combination has no concurrency problem to solve (Modules 03, 04). **Most people reaching for lock-free should stop here.**
- 2 Can you shard the structure?** Many locks over disjoint partitions — a lock per hash bucket, a queue per thread — scales nearly as well as lock-free and is far simpler to get right.
- 3 Have you measured the mutex?** It is frequently not the bottleneck that was assumed. An uncontended mutex costs tens of nanoseconds, and if the critical section is longer than that the lock is not the problem.

- 4 **Do you need the progress guarantee, or do you want speed?** These are different requirements with different answers, and conflating them is how people end up with slower, harder code.
- 5 **Is there a tested library implementation?** There usually is.

#### USE A LIBRARY

**Correct lock-free data structures are published as research papers, and several well-known published algorithms had bugs discovered years after publication** — by people specifically looking, with model checkers.

**The memory model subtleties, the ABA cases, and the reclamation scheme each admit many ways to be subtly wrong**, and the resulting failures share the worst possible combination of properties: rare, non-reproducible, architecture-dependent, and catastrophic.

**Use a tested implementation.** Boost.Lockfree, Facebook's folly, libcds, Java's java.util.concurrent, or your platform's concurrent collections. These have been exercised by far more code than yours will be.

**Write your own in order to understand them**, which is what this module and its exercises are for. **Ship someone else's.** That is not a lack of ambition; it is the correct engineering judgement for a component whose failure mode is silent corruption.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                                   | What it covers for this module                                                                                                          |
|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Herlihy & Shavit — The Art of Multiprocessor Programming                                   | The standard text. Progress guarantees, CAS, and the classic lock-free structures, with proofs.                                         |
| Michael — Hazard Pointers (free)                                                           | The reclamation scheme of §3, in the original.                                                                                          |
| McKenney — Is Parallel Programming Hard, And, If So, What Can You Do About It? (free book) | <b>Free, enormous, and excellent</b> , especially on RCU, which the author designed. The honest-judgement material in §4 is throughout. |
| Fedor Pikus — talks on lock-free programming (free)                                        | Practical, measured, and unusually honest about when lock-free loses.                                                                   |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a lock-free stack with CAS. Verify it with a stress test and ThreadSanitizer.

- 2 Measure it against a mutex-protected stack at 1, 2, 4, 8, and 32 threads. **Plot both and find where lock-free loses.**
- 3 Measure both against per-thread stacks combined at the end.
- 4 **Construct an ABA failure** deliberately, with a slow thread and a recycled node. This requires care and is instructive.
- 5 Fix it with a tagged pointer and confirm the failure disappears.
- 6 Implement hazard pointers for your stack and measure the read-side cost.
- 7 Implement epoch-based reclamation and compare the read cost and the memory high-water mark against hazard pointers.
- 8 Replace reclamation entirely with a type-stable pool and compare.
- 9 Benchmark your implementation against Boost.Lockfree. Report the gap honestly.
- 10 For a sharing problem in your Project 1 code, work through the five questions of §4 and document the decision.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define wait-free, lock-free, obstruction-free, and blocking.
- 2 Why is lock-free not a speed claim, and where does the guarantee genuinely matter?
- 3 Write a CAS loop and say when to use weak rather than strong.
- 4 Explain the ABA problem and give three solutions.
- 5 Why can you not simply free an unlinked node, and why is this the hard part?
- 6 Compare four reclamation schemes on mechanism and cost.
- 7 Give the five questions to ask before writing lock-free code.
- 8 Why is using a library the right recommendation?